МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИКН)

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Сетевые технологии»

Отчёт по лабораторной работе на тему:

«ЛР-2. Команды тестирования и исправления настроек сети»

Выполнил: студент группы БИВТ-24-7

Бут Иван Антонович

Проверил: КТН, СНС, доцент

Микитенко Игорь Иванович

МОСКВА 2026

Цель работы

Получить представление о тестировании и восстановлении подключений и доступа к узлам в ip-сетях, маршрутах прохождения сигналов, анализа состояния портов, сбора сетевой статистики и получения другой информации.

Основные задачи лабораторной работы

1. TRACERT: определить маршрут до удаленного узла, посмотреть задержки и разобраться, что означает число прыжков.
2. PATHPING: сравнить трассировку и статистику потерь до habinfo.ru и cdek.ru.
3. GETMAC: определить MAC-адреса интерфейсов и сравнить Wi-Fi, Ethernet и виртуальные адаптеры.
4. NSLOOKUP: найти **IP-адрес** роутера и понять, почему локальный DNS-сервер может определяться как "UnKnown".
5. NETSTAT: разобрать активные подключения, открытые порты и сетевую статистику.
6. NETSH: посмотреть настройки файрвола, сведения о WLAN и попробовать команды для виртуальной точки доступа.
7. NET: поработать со службами, общими ресурсами, сетевыми дисками и локальными группами.
8. ROUTE.EXE: посмотреть таблицу маршрутизации и понять смысл метрики.
9. TELNET: изучить параметры клиента и проверить подключение к разным портам.
10. Файловые команды: создать, скопировать, переместить, переименовать и удалить файлы и папки из консоли.

МОСКВА 2026

Ход выполнения работы

Работа выполнялась в Windows в командной строке и PowerShell. Я шел по заданиям по порядку, а для дополнительных пунктов отдельно собрал расширенные логи. Ниже все оформлено последовательно: сначала формулировка задания, потом коротко что именно было сделано, дальше идут логи и в конце вывод по каждому пункту.

Задание 1. Команда TRACERT

Здесь требовалось выполнить трассировку маршрута до узла сети Интернет, определить количество промежуточных маршрутизаторов и оценить задержки на каждом переходе. В качестве цели я взял публичные DNS, потому что они стабильно доступны и хорошо показывают маршрут.

Логи

```
Tracing route to one.one.one.one [1.1.1.1] over a maximum of 30 hops:
1 1 ms 1 ms 1 ms XiaoQiang [192.168.31.1]
2 5 ms 2 ms 2 ms 172.19.63.252
3 5 ms 2 ms 2 ms 10.200.40.9
4 5 ms 3 ms 3 ms 10.200.40.10
5 5 ms 2 ms 2 ms 176x213x132x18.static-business.msk.ertelecom.ru [176.213.132.18]
6 33 ms 34 ms 54 ms 188x234x72x50.dynamic.omsk.ertelecom.ru [188.234.72.50]
7 58 ms 44 ms 58 ms 188x234x131x45.ertelecom.ru [188.234.131.45]
8 59 ms 50 ms 48 ms 162.158.236.11
9 34 ms 34 ms 34 ms one.one.one.one [1.1.1.1]
Trace complete.
```

Дополнительная трассировка до другого публичного DNS:

```
Tracing route to dns9.quad9.net [9.9.9.9] over a maximum of 30 hops:
1 1 ms 1 ms 1 ms XiaoQiang [192.168.31.1]
2 6 ms 3 ms 3 ms 172.19.63.252
3 5 ms 2 ms 2 ms 10.200.40.9
4 5 ms 2 ms 2 ms 10.200.40.10
5 3 ms 2 ms 2 ms 176x213x132x18.static-business.msk.ertelecom.ru [176.213.132.18]
6 6 ms 4 ms 3 ms ertelekom-ic-389478.ip.twelve99-cust.net [62.115.145.221]
7 4 ms 3 ms 4 ms mow-b2-link.ip.twelve99.net [62.115.145.220]
8 26 ms 25 ms 25 ms sto-bb1-link.ip.twelve99.net [62.115.143.24]
9 26 ms 26 ms 26 ms sto-b1-link.ip.twelve99.net [62.115.125.211]
10 26 ms 26 ms 26 ms i3dnet-ic-369293.ip.twelve99-cust.net [62.115.166.43]
11 26 ms 26 ms 23 ms hosted-by.i3d.net [109.200.195.196]
12 23 ms 25 ms 25 ms dns9.quad9.net [9.9.9.9]
Trace complete.
```

Пример ограничения числа прыжков (параметр -h) для быстрой проверки ближайших узлов:

```
Tracing route to dns.google [8.8.8.8] over a maximum of 5 hops:
1 1 ms 1 ms 1 ms XiaoQiang [192.168.31.1]
2 5 ms 2 ms 2 ms 172.19.63.252
3 5 ms 2 ms 2 ms 10.200.40.9
4 4 ms 2 ms 2 ms 10.200.40.10
```

```
5 7 ms 20 ms 5 ms 176x213x132x18.static-business.msk.ertelecom.ru [176.213.132.18]
Trace complete.
```

Контрольный вопрос: "Число прыжков" это количество промежуточных маршрутизаторов, через которые проходит пакет до цели. Каждая строка соответствует одному hop и показывает задержки ICMP-ответов для этого узла.

Наблюдение: на маршруте до 1.1.1.1 заметен рост RTT на 6-8 hop, где задержки возрастают до десятков миллисекунд. Обычно это момент выхода из локальной сети провайдера на магистральные узлы.

TRACERT показывает маршрут и дает быструю оценку задержек. Скачки RTT чаще всего возникают на магистральных участках и на стыках между сетями операторов.

Задание 2. Команда PATHPING

В этом пункте нужно было выполнить PATHPING до habinfo.ru и cdek.ru. Команда похожа на TRACERT, но дополнительно собирает статистику потерь, поэтому выполняется заметно дольше.

Логи

PATHPING до habinfo.ru:

```
echo START %time% & pathping habinfo.ru & echo END %time%
START 0:02:39.78
Tracing route to habinfo.ru [185.177.219.148] over a maximum of 30 hops:
0 Study_Ivan [192.168.31.15]
1 XiaoQiang [192.168.31.1]
2 172.19.63.252
3 10.200.40.9
4 10.200.40.10
5 176x213x132x18.static-business.msk.ertelecom.ru [176.213.132.18]
6 bbr01.nsk.ertelecom.ru [188.234.129.211]
7 nsk-ix.adman.com [193.232.87.32]
8 nsk-mx5.adman.com [185.133.41.61]
9 185.177.219.148
Computing statistics for 225 seconds...
Source to Here This Node/Link
Hop RTT Lost/Sent = Pct Lost/Sent = Pct Address
0 Study_Ivan [192.168.31.15]
1 1ms 0/100=0% 0/100=0% XiaoQiang [192.168.31.1]
2 --- 100/100=100% 100/100=100% 172.19.63.252
6 54ms 0/100=0% 0/100=0% bbr01.nsk.ertelecom.ru [188.234.129.211]
8 55ms 0/100=0% 0/100=0% nsk-mx5.adman.com [185.133.41.61]
9 54ms 0/100=0% 0/100=0% 185.177.219.148
Trace complete.
END 0:02:39.78
```

PATHPING до cdek.ru:

```
echo START %time% & pathping cdek.ru & echo END %time%
START 0:07:52.40
Tracing route to cdek.ru [155.212.234.126] over a maximum of 30 hops:
0 Study_Ivan [192.168.31.15]
1 XiaoQiang [192.168.31.1]
2 172.19.63.252
3 10.200.40.9
4 10.200.40.10
5 176x213x132x18.static-business.msk.ertelecom.ru [176.213.132.18]
6 10.0.199.181
7 10.0.199.180
8 10.10.0.243
9 * * *
Computing statistics for 200 seconds...
...
END 0:07:52.40
```

Пояснение: 100% потерь на промежуточных hop в PATHPING часто связаны не с реальной потерей трафика, а с фильтрацией ICMP на маршрутизаторах. При этом конечный узел может отвечать без потерь.

PATHPING полезен для поиска проблемных участков, но результаты по промежуточным узлам нужно интерпретировать осторожно и не путать фильтрацию ICMP с обрывом передачи данных.

Задание 3. Команда GETMAC

Требовалось определить MAC-адреса сетевых интерфейсов и сравнить их. Я сначала проверил свой IP через IPCONFIG, потом вывел MAC адреса через GETMAC.

Логи

```
ipconfig
IPv4 Address: 192.168.31.15
Default Gateway: 192.168.31.1
getmac /v
Connection Name | Network Adapter | Physical Address
WiFi | Intel(R) Wi-Fi | F2-12-DF-A3-11-FA
Ethernet | Realtek USB GbE | 6C-1F-F7-13-42-AD
VMware Network Adapter VMnet1 | 00-50-56-C0-00-01
VMware Network Adapter VMnet8 | 00-50-56-C0-00-08
getmac /s ya.ru /v
ERROR: The RPC server is unavailable.
```

Сравнение: в системе есть Ethernet, WiFi и виртуальные адаптеры, поэтому MAC-адресов несколько. Например, Ethernet адаптер имеет MAC 6C-1F-F7-13-42-AD, а WiFi адаптер имеет MAC F2-12-DF-A3-11-FA.

Попытка получить MAC удаленного узла "ya.ru" закончилась ошибкой RPC. Это нормально, потому что GETMAC /s работает через RPC и рассчитан на локальную сеть или домен, где удаленные вызовы разрешены и есть доступ.

GETMAC стабильно работает для локальных интерфейсов. Для узлов в Интернете этот способ, как правило, не подходит.

Задание 4. Команда NSLOOKUP

Нужно было определить **IP-адреса** роутера и объяснить, почему имя DNS-сервера может отображаться как "UnKnown". В моем случае **IP-адрес** роутера это 192.168.31.1, а имя устройства в ответе это XiaoQiang. Я проверил роутер по адресу шлюза и сделал несколько запросов к доменным именам.

Логи

Определение **IP-адреса** роутера:

```
nslookup 192.168.31.1
Server: UnKnown
Address: fe80::8ede:f9ff:fee4:d318
Name: XiaoQiang
Address: 192.168.31.1
```

Пример запросов к публичным именам и к заданному DNS-серверу:

```
nslookup mail.ru
Server: UnKnown
Address: fe80::8ede:f9ff:fee4:d318
Non-authoritative answer:
Name: mail.ru
Addresses: 89.221.239.1 185.180.201.1 90.156.232.4
nslookup google.ru 1.1.1.1
Server: one.one.one.one
Address: 1.1.1.1
Non-authoritative answer:
Name: google.ru
Addresses: 2a00:1450:4010:c06::5e 64.233.163.94
```

Почему "UnKnown": DNS-сервер отображается по link-local IPv6 адресу fe80::..., а обратная DNS-запись для такого адреса часто не настроена на домашнем роутере. Поэтому имя не определяется, хотя сервер отвечает и выполняет резолвинг.

NSLOOKUP подходит для проверки DNS и для подтверждения адреса роутера. "UnKnown" это чаще всего отсутствие reverse DNS, а не проблема DNS.

Задание 5. Команда NETSTAT

По заданию требовалось собрать информацию об активных подключениях и открытых портах. В логах к этому пункту дополнительно зафиксированы сетевые параметры (IP, шлюз и DNS), чтобы было понятно, в какой сети выполнялся анализ.

Логи

Сетевые параметры:

```
ipconfig /all
IPv4 Address: 192.168.31.15
Default Gateway: 192.168.31.1
DNS Servers: fe80::8ede:f9ff:fee4:d318, 192.168.31.1
```

Ответ на контрольный вопрос о портах: типичные сервисные порты это 80 (HTTP), 443 (HTTPS), 22 (SSH), 23 (TELNET), 53 (DNS), 25 (SMTP). В выводе NETSTAT порты отображаются как локальные и удаленные точки соединений TCP/UDP.

Практический алгоритм: выполнить "netstat -ano" и при необходимости отфильтровать вывод, затем по PID определить процесс, который держит порт.

NETSTAT является базовым инструментом анализа портов и соединений. Для интерпретации выводов важно понимать назначение распространенных портов и уметь связывать запись NETSTAT с процессом по PID.

Задание 6. Команда NETSH

Требовалось изучить контексты NETSH и выполнить примеры работы с файрволом и WLAN. Честно, на этом пункте я пару минут ругался на Windows, потому что не ожидал, что список WiFi сетей завязан на разрешения приватности.

Логи

```
netsh advfirewall show allprofiles
State ON
Firewall Policy BlockInbound,AllowOutbound
...
netsh wlan show networks mode=bssid
Network shell commands need location permission to access WLAN information.
Access is denied.
```

```
netsh wlan show drivers
Driver: Intel(R) Wi-Fi 6 AX201 160MHz
Radio types supported: 802.11b 802.11g 802.11n 802.11a 802.11ac 802.11ax
netsh wlan set hostednetwork mode=allow ssid=LR2_Test key=Password123
The hosted network mode has been set to allow.
netsh wlan start hostednetwork
The hosted network couldn't be started. The group or resource is not in the correct
state ...
```

Комментарий: профили файрвола включены. При попытке показать доступные сети WiFi команда потребовала включить геолокацию и выдала отказ в доступе. Также выполнена настройка hostednetwork, но запуск точки доступа не удался из-за состояния ресурса.

NETSH дает широкий доступ к настройкам, но часть функций зависит от прав администратора и системных разрешений в Windows.

Задание 7. Команда NET и работа со службами и общими ресурсами

Нужно было управлять сетевыми службами и создать общий ресурс. В этом задании видно, что администрирование в Windows сильно зависит от прав пользователя.

Логи

Управление службами:

```
sc query dnscache
SERVICE_NAME: dnscache STATE: 4 RUNNING (NOT_STOPPABLE, NOT_PAUSABLE,
IGNORES_SHUTDOWN)
net stop dnscache
The requested pause, continue, or stop is not valid for this service.
sc query schedule
SERVICE_NAME: schedule STATE: 4 RUNNING (STOPPABLE ...)
net stop schedule
System error 5 has occurred. Access is denied.
sc query w32time
SERVICE_NAME: w32time STATE: 4 RUNNING
net start w32time
The Windows Time service was started successfully.
```

Создание общего ресурса и подключение сетевого диска:

```
net share LR2Share=C:\...\Share
LR2Share was shared successfully.
net share LR2Share$=C:\...\Share
LR2Share$ was shared successfully.
net use Z: \\STUDY_IVAN\LR2Share$
The command completed successfully.
Status Local Remote Network
OK Z: \\STUDY IVAN\LR2Share$ Microsoft Windows Network
net use Z: /delete
Z: was deleted successfully.
```

Комментарий: "dnscache" оказался неостанавливаемой службой, что прямо видно по флагу NOT_STOPPABLE. Ошибка доступа при остановке

"schedule" указывает на отсутствие прав администратора. Общий ресурс LR2Share$ создан и подключен как диск Z:.

SC и NET позволяют управлять службами и общими ресурсами, но результат зависит от прав и настроек безопасности.

Задание 8. Утилита ROUTE.EXE

Задача: вывести таблицу маршрутизации и объяснить "метрику". Я посмотрел активные маршруты и сверил параметры ROUTE по справке.

Логи

```
route print
Network Destination Netmask Gateway Interface Metric
0.0.0.0 0.0.0.0 192.168.31.1 192.168.31.15 25
...
Persistent Routes: None
```

Параметры ROUTE:

```
route /?
ROUTE [-f] [-p] [-4|-6] command [destination] [MASK netmask] [gateway] [METRIC
metric] [IF interface]
METRIC specifies the metric, ie. cost for the destination.
Example: route PRINT -4
```

Метрика: числовая стоимость маршрута. ОС выбирает маршрут с наименьшей метрикой среди подходящих записей, поэтому метрика прямо влияет на выбор пути трафика.

ROUTE помогает диагностировать выбор пути в сети. Если метрики настроены неудачно, трафик может уходить не через тот интерфейс, который ожидается.

Задание 9. Утилита TELNET

Задание: изучить параметры TELNET и провести тесты соединений по портам. Сейчас TELNET чаще используют как проверку доступности TCP-порта, а не как средство удаленного доступа.

Логи

Справка по TELNET:

```
telnet /?
telnet [-a][-e escape char][-f log file][-l user][-t term][host [port]]
```

```
-f File name for client side logging
-t Specifies terminal type (vt100, vt52, ansi, vtnt)
host Specifies the hostname or IP address of the remote computer to connect to
port Specifies a port number or service name
```

Комментарий: в репозитории есть файлы с TELNET тестами, но они содержат управляющие символы терминала, поэтому в текстовом отчете они отображаются нестабильно. Логика проверки простая: подключиться к host:port и посмотреть, установилось ли TCP-соединение.

TELNET подходит для быстрых тестов портов, но протокол небезопасен для администрирования, так как передает данные без шифрования. Поэтому его заменяют SSH и HTTPS.

Задание 10. Файловые команды в консоли

В этом пункте я уже не смотрел на сеть напрямую, а отрабатывал обычные консольные действия с файлами и папками. По факту это тоже полезная часть работы: когда логов много, очень удобно быстро создавать структуру каталогов, складывать туда результаты и искать нужные строки прямо из консоли.

Логи

Сначала я создал отдельную рабочую папку и простую вложенную структуру каталогов. Команда mkdir нужна для создания папок. Здесь было важно не просто сделать одну директорию, а сразу собрать небольшое дерево A\B\C, чтобы потом на нем проверить копирование и перемещение.

```
mkdir "sandbox_cmd"
mkdir A
mkdir A\B
mkdir A\B\C
```

Дальше я создал два текстовых файла. Команда echo позволила сразу записать строку в файл из консоли. После этого командой copy я скопировал первый файл в каталог A, а командой xcopy продублировал уже целую структуру каталогов целиком. Это удобнее, когда внутри есть вложенные папки и их нужно перенести без ручной сборки.

```
echo First file> file1.txt
```

```
echo Second one> file2.txt
copy file1.txt A\
xcopy A A_copy /E /I /Y
```

Следующим шагом я проверил перемещение, переименование и удаление. Команда move переносит файл в другой каталог, ren меняет имя файла, del удаляет файл, а findstr ищет нужный текст по содержимому. То есть в одном задании получилось отработать почти весь базовый набор действий, который реально часто нужен при разборе логов.

```
move file2.txt A\B\
ren file1.txt file1_renamed.txt
del A\B\file2.txt
findstr "First" A\file1.txt
```

После выполнения команд я отдельно посмотрел содержимое каталогов. Команда dir показала, что файл file1.txt действительно лежит в A, а до удаления file2.txt находился в A\B. Поиск через findstr тоже отработал корректно и нашел строку First file в созданном файле.

```
dir A
file1.txt
dir A\B
file2.txt
findstr "First" A\file1.txt
First file
```

По этому заданию я для себя еще раз зафиксировал простую вещь: даже без графического интерфейса можно быстро собрать рабочую структуру, разложить файлы по папкам, переименовать их и сразу проверить результат. Особенно удобно это в лабораторных, когда не хочется вручную открывать каждую папку и искать, куда что переместилось.

Отдельно стало видно, чем отличаются команды copy и xcopy. Первая подходит для обычного копирования файла, а вторая уже удобна для папок и вложенной структуры. findstr тоже оказался полезным, потому что с ним не надо открывать файл ради одной строки: достаточно сразу проверить, есть ли нужный текст в содержимом.

Дополнительное задание 1. Трассировка маршрута с анализом задержек

Цель: определить маршруты до удаленных узлов, сравнить задержки на участках и понять, где появляются "Request timed out". В ходе работы я использовал только Windows, поэтому сравнение с Linux или macOS описываю теоретически, без отдельного листинга.

Логи

Шаг 1. Базовая трассировка до узла Google по IPv4 с резолвингом имен.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 142.250.72.206
Tracing route to ncdena-ab-in-f14.1e100.net [142.250.72.206]
over a maximum of 30 hops:
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     6 ms     2 ms     2 ms  10.200.40.10
5     6 ms     2 ms     2 ms  176x213x132x18.static-business.msk.ertelecom.ru
[176.213.132.18]
6     6 ms     3 ms     3 ms  72.14.215.165
7    15 ms     *       16 ms  188x234x131x225.ertelecom.ru [188.234.131.225]
8    16 ms    16 ms    16 ms  216.239.49.19
9    17 ms    18 ms    17 ms  192.178.241.234
10    31 ms    31 ms    32 ms  142.250.238.14
11    42 ms    40 ms    39 ms  216.239.63.63
12   119 ms   122 ms   122 ms  192.178.80.59
13   135 ms   134 ms   135 ms  192.178.81.233
14   145 ms   144 ms   145 ms  192.178.255.14
15   145 ms   146 ms   143 ms  172.253.70.31
16   154 ms   153 ms   154 ms  142.251.237.111
17   154 ms   160 ms   153 ms  172.253.51.121
18   154 ms   154 ms   153 ms  192.178.97.21
19   154 ms   153 ms   153 ms  142.251.51.155
20   153 ms   153 ms   151 ms  ncdena-ab-in-f14.1e100.net [142.250.72.206]
Trace complete.
```

Краткий комментарий: на первых прыжках виден домашний роутер и узлы провайдера. Дальше появляются магистральные узлы, и задержка растет заметнее после межрегиональных участков.

Шаг 2. Трассировка с флагами -d, -h, -w. Я специально ускорил вывод, отключив резолвинг имен, и увеличил лимит прыжков и таймаут.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -d -h 60 -w 350 142.250.72.206
Tracing route to 142.250.72.206 over a maximum of 60 hops
1     1 ms     1 ms     1 ms  192.168.31.1
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     5 ms     2 ms     2 ms  10.200.40.10
5     6 ms     2 ms     3 ms  176.213.132.18
6     6 ms     3 ms     2 ms  72.14.215.165
7    16 ms    16 ms    16 ms  188.234.131.225
8    16 ms    15 ms     *     216.239.49.19
9    17 ms    16 ms    16 ms  192.178.241.234
10     *       30 ms    31 ms  142.250.238.14
```

```
11    41 ms    41 ms    41 ms  216.239.63.63
12   120 ms   121 ms   121 ms  192.178.80.59
13     *      134 ms   131 ms  192.178.81.233
14   144 ms   143 ms     *     192.178.255.14
15   144 ms   145 ms   145 ms  172.253.70.31
16   153 ms     *      151 ms  142.251.237.111
17   174 ms   153 ms     *     172.253.51.121
18   154 ms   153 ms   153 ms  192.178.97.21
19   152 ms     *      153 ms  142.251.51.155
20   152 ms   152 ms   152 ms  142.250.72.206
Trace complete.
```

Наблюдение: при "-d" имена не выводятся, зато трассировка идет быстрее. По значениям мс видно, что общая картина по задержкам остается похожей.

Шаг 3. Проверка поведения при попытке использовать IPv6 флаг для IPv4 адреса.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -6 -h 60 -w 350 142.250.72.206
Unable to resolve target system name 142.250.72.206.
```

Вывод: флаг "-6" ожидает IPv6 цель. Если передать IPv4 адрес, tracert не сможет корректно определить целевую систему в IPv6 контексте.

Шаг 4. Маршрут до удаленного узла в Азии. В логах цель 8.218.56.1, дальше трасса уходит через Европу и выходит в Гонконг, после чего начинаются таймауты.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 8.218.56.1
Tracing route to 8.218.56.1 over a maximum of 30 hops
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     3 ms     3 ms  10.200.40.9
4     5 ms     3 ms     3 ms  10.200.40.10
5     5 ms     3 ms     2 ms  176x213x132x18.static-business.msk.ertelecom.ru
[176.213.132.18]
6     3 ms     3 ms     3 ms  ertelekom-ic-389478.ip.twelve99-cust.net
[62.115.145.221]
7     *       13 ms    16 ms  ae15-1316.RT.M9.MSK.retn.ru [139.45.224.66]
8    46 ms    42 ms    42 ms  ae1-5.rt.irx.fkt.de.retn.net [87.245.232.244]
9    45 ms    45 ms    46 ms  xe-5-2-2.a03.frnkge13.de.bb.gin.ntt.net
[83.231.214.76]
10    56 ms    54 ms    51 ms  ae-3.r26.frnkge13.de.bb.gin.ntt.net [129.250.4.174]
11    60 ms    63 ms    57 ms  ae-3.r22.vienat02.at.bb.gin.ntt.net [129.250.7.34]
12     *        *      311 ms  ae-9.r28.tkokhk01.hk.bb.gin.ntt.net [129.250.4.80]
13   265 ms   248 ms   235 ms  ae-18.a00.chwahk03.hk.bb.gin.ntt.net [129.250.2.149]
14   271 ms   270 ms   273 ms  203.131.242.186
15     *        *        *     Request timed out.
16     *        *        *     Request timed out.
17     *      269 ms     *     26.25.157.1
18     *        *        *     Request timed out.
19     *        *        *     Request timed out.
20     *        *        *     Request timed out.
21     *        *        *     Request timed out.
22     *        *        *     Request timed out.
23     *        *        *     Request timed out.
24     *        *        *     Request timed out.
25     *        *        *     Request timed out.
26     *        *        *     Request timed out.
```

```
27     *        *        *     Request timed out.
28     *        *        *     Request timed out.
29     *        *        *     Request timed out.
30     *        *        *     Request timed out.
Trace complete.
```

Шаг 5. Та же цель, но с увеличенным лимитом прыжков и таймаутом. Это показывает, что после определенного узла многие маршрутизаторы перестают отвечать на ICMP TTL expired, хотя трафик в целом может идти дальше.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -h 100 -w 500 8.218.56.1
Tracing route to 8.218.56.1 over a maximum of 100 hops
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     6 ms     3 ms     2 ms  172.19.63.252
3     4 ms     2 ms     2 ms  10.200.40.9
4     2 ms     2 ms     2 ms  10.200.40.10
5     3 ms     2 ms     2 ms  176x213x132x18.static-business.msk.ertelecom.ru
[176.213.132.18]
6    15 ms     5 ms     5 ms  ertelekom-ic-389478.ip.twelve99-cust.net
[62.115.145.221]
7     *       16 ms    15 ms  ae15-1316.RT.M9.MSK.retn.ru [139.45.224.66]
8    42 ms    42 ms    42 ms  ae1-5.rt.irx.fkt.de.retn.net [87.245.232.244]
9    43 ms    43 ms    43 ms  xe-5-2-2.a03.frnkge13.de.bb.gin.ntt.net
[83.231.214.76]
10    46 ms    46 ms    45 ms  ae-3.r26.frnkge13.de.bb.gin.ntt.net [129.250.4.174]
11    57 ms    56 ms     *     ae-3.r22.vienat02.at.bb.gin.ntt.net [129.250.7.34]
12     *        *        *     Request timed out.
13   361 ms     *      373 ms  ae-18.a00.chwahk03.hk.bb.gin.ntt.net [129.250.2.149]
14   301 ms     *      303 ms  203.131.242.186
15     *        *        *     Request timed out.
16     *        *        *     Request timed out.
17     *        *        *     Request timed out.
18     *        *        *     Request timed out.
98     *        *        *     Request timed out.
99     *        *        *     Request timed out.
100     *        *        *     Request timed out.
Trace complete.
Еще раз протестил
```

Шаг 6. Попытка ускорить трассировку к той же цели через -d (без имен).

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -d -h 60 -w 350 8.218.56.1
Tracing route to 8.218.56.1 over a maximum of 60 hops
1     1 ms     1 ms     1 ms  192.168.31.1
2     5 ms     3 ms     3 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     3 ms     3 ms     3 ms  10.200.40.10
5     5 ms     2 ms     2 ms  176.213.132.18
6     7 ms     5 ms     3 ms  62.115.145.221
7    16 ms    13 ms    15 ms  139.45.224.66
8    43 ms    45 ms    45 ms  87.245.232.244
9    46 ms    46 ms    46 ms  83.231.214.76
10    46 ms    45 ms     *     129.250.4.174
11    57 ms    57 ms    57 ms  129.250.7.34
12     *        *        *     Request timed out.
13   364 ms     *      342 ms  129.250.2.149
14   299 ms     *      282 ms  203.131.242.186
15     *        *        *     Request timed out.
16     *        *        *     Request timed out.
(Тоже дальше смысла нет)
```

Шаг 7. Трассировка внутри РФ: домен ya.ru, а также явный IPv4 адрес Yandex DNS 77.88.44.242, сначала без имен, потом с именами.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 ya.ru
Tracing route to ya.ru [77.88.44.242]
over a maximum of 30 hops:
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     5 ms     2 ms     2 ms  10.200.40.10
5     5 ms     2 ms     2 ms  176x213x132x18.static-business.msk.ertelecom.ru
[176.213.132.18]
6     5 ms     2 ms     2 ms  188x234x76x65.ertelecom.ru [188.234.76.65]
7     8 ms     4 ms     3 ms  169.254.0.115
8     9 ms     7 ms     6 ms  10.32.83.1
9     9 ms     6 ms     6 ms  10.1.3.1
10     8 ms     5 ms     5 ms  ya.ru [77.88.44.242]
Trace complete.
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -d -h 60 -w 350 77.88.44.242
Tracing route to 77.88.44.242 over a maximum of 60 hops
1     1 ms     1 ms     1 ms  192.168.31.1
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     6 ms     2 ms     2 ms  10.200.40.10
5    24 ms    22 ms     5 ms  176.213.132.18
6     5 ms     2 ms     2 ms  188.234.76.65
7     5 ms     3 ms     3 ms  169.254.0.115
8     9 ms     6 ms     5 ms  10.32.83.1
9     9 ms     6 ms     6 ms  10.1.3.1
10     8 ms     5 ms     5 ms  77.88.44.242
Trace complete.
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -h 60 -w 350 77.88.44.242
Tracing route to ya.ru [77.88.44.242]
over a maximum of 60 hops:
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     6 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     6 ms     2 ms     2 ms  10.200.40.10
5     6 ms     2 ms     2 ms  176x213x132x18.static-business.msk.ertelecom.ru
[176.213.132.18]
6     6 ms     2 ms     2 ms  188x234x76x65.ertelecom.ru [188.234.76.65]
7     6 ms     3 ms     3 ms  169.254.0.115
8     8 ms     5 ms     5 ms  10.32.83.1
9    10 ms     7 ms     7 ms  10.1.3.1
10     7 ms     5 ms     5 ms  ya.ru [77.88.44.242]
Trace complete.
```

Шаг 8. Дальний маршрут до Австралии по IPv4. В этих тестах я менял параметры и сравнивал, где появляются потери.

```
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -h 60 -w 400 220.245.143.194
Tracing route to 220-245-143-194.tpgi.com.au [220.245.143.194]
over a maximum of 60 hops:
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     4 ms     2 ms     2 ms  10.200.40.10
5     2 ms     2 ms     2 ms  176x213x132x18.static-business.msk.ertelecom.ru
[176.213.132.18]
6     6 ms     2 ms     2 ms  ertelekom-ic-389478.ip.twelve99-cust.net
[62.115.145.221]
7     *        *       15 ms  ae15-1316.RT.M9.MSK.retn.ru [139.45.224.66]
8    47 ms    43 ms    43 ms  ae0-13.rt.ldc.waw.pl.retn.net [87.245.233.56]
```

```
9    41 ms    38 ms    40 ms  ix-ae-35-0.tcore1.w1t-warsaw.as6453.net
[80.231.124.38]
10     *        *        *     Request timed out.
11     *        *        *     Request timed out.
12     *        *        *     Request timed out.
13   179 ms   176 ms     *     if-bundle-12-2.qcore1.pvu-paris.as6453.net
[80.231.245.12]
14   180 ms   181 ms     *     if-bundle-22-2.qcore1.pye-paris.as6453.net
[80.231.154.198]
15     *        *      178 ms  63.243.137.148
16     *        *        *     Request timed out.
17   349 ms     *      348 ms  220-245-143-194.tpgi.com.au [220.245.143.194]
Trace complete.
C:\Users\Ivan\MISoS\4 семестр\Клиент-серверные приложения и сетевые
технологии\сети\2 лаба>tracert -4 -d -h 60 -w 400 220.245.143.194
Tracing route to 220.245.143.194 over a maximum of 60 hops
1     1 ms     1 ms     1 ms  192.168.31.1
2     5 ms     2 ms     2 ms  172.19.63.252
3     4 ms     2 ms     2 ms  10.200.40.9
4     5 ms     2 ms     2 ms  10.200.40.10
5     5 ms     2 ms     2 ms  176.213.132.18
6     6 ms     2 ms     2 ms  62.115.145.221
7    15 ms    12 ms    15 ms  139.45.224.66
8    45 ms    42 ms    42 ms  87.245.233.56
9    41 ms     *       39 ms  80.231.124.38
10   179 ms     *        *     195.219.69.118
11     *        *        *     Request timed out.
12     *      179 ms     *     80.231.245.40
13   175 ms   179 ms     *     80.231.245.12
14     *        *        *     Request timed out.
15     *        *      178 ms  63.243.137.148
16   177 ms     *        *     207.45.219.136
17   363 ms     *      352 ms  220.245.143.194
Trace complete.
```

Шаг 9. Дополнительная трассировка из другого листинга (Kazan additional). Это полезно как сравнение маршрута и задержек в другой точке доступа.

```
C:\Windows\System32>tracert -4 -h 60 -w 400 220.245.143.194
Трассировка маршрута к 194.143.245.220.in-addr.arpa [220.245.143.194]
с максимальным числом прыжков 60:
1    <1 мс    <1 мс    <1 мс  rt [192.168.0.1]
2     2 ms     1 ms     1 ms  100.97.0.1
3     1 ms     1 ms     1 ms  188.254.26.240
4     1 ms     1 ms     1 ms  188.254.2.221
5    33 ms    33 ms    33 ms  188.128.126.95
6     *        *        *     Превышен интервал ожидания для запроса.
7    34 ms    34 ms    34 ms  be4649.ccr22.sto03.atlas.cogentco.com [130.117.3.130]
8    44 ms    43 ms    44 ms  be2557.rcr21.cph01.atlas.cogentco.com [154.54.61.241]
9     *       54 ms    54 ms  be2496.ccr41.ham01.atlas.cogentco.com [154.54.61.221]
10    61 ms    60 ms    60 ms  be2815.ccr41.ams03.atlas.cogentco.com [154.54.38.205]
11     *      193 ms   156 ms  be3530.ccr81.lon05.atlas.cogentco.com [154.54.75.153]
12    65 ms    64 ms    64 ms  port-channel3464.ccr91.lhr01.atlas.cogentco.com
[154.54.75.150]
13   155 ms   155 ms     *     be8671.ccr41.jfk02.atlas.cogentco.com [154.54.167.69]
14   149 ms   149 ms     *     port-channel4985.ccr91.cle04.atlas.cogentco.com
[154.54.162.165]
15   150 ms   149 ms     *     be2717.ccr41.ord01.atlas.cogentco.com [154.54.6.221]
16   176 ms   176 ms     *     be5214.ccr31.oma02.atlas.cogentco.com
[154.54.165.133]
17     *        *        *     Превышен интервал ожидания для запроса.
18   182 ms   183 ms     *     be2353.ccr81.slc03.atlas.cogentco.com [154.54.5.102]
19   197 ms   197 ms     *     be3905.ccr21.sfo01.atlas.cogentco.com [154.54.1.210]
20   202 ms   201 ms     *     be3096.ccr81.sjc13.atlas.cogentco.com [154.54.40.41]
21   197 ms   197 ms     *     be2664.rcr71.sjc14.atlas.cogentco.com [154.54.88.230]
22   363 ms     *      363 ms  Hu0-4-0-0.rcr21.b001848-1.sjc01.atlas.cogentco.com
[38.32.115.106]
23   376 ms     *      376 ms  syd-apt-ros-crt3-be-100.tpgi.com.au [203.29.134.43]
```

```
24   364 ms     *      364 ms  syd-gls-har-crt3-be-30.tpg.com.au [203.29.135.105]
25   366 ms     *      365 ms  syd-gls-har-bras220-po-10.tpgi.com.au [203.221.3.28]
26     *      382 ms     *     194.143.245.220.in-addr.arpa [220.245.143.194]
27   383 ms     *      382 ms  194.143.245.220.in-addr.arpa [220.245.143.194]
Трассировка завершена.
```

Пояснение по ключевым флагам TRACERT, которые использовались в тестах:

-4: принудительно использовать IPv4.

-6: принудительно использовать IPv6.

-d: не выполнять DNS-резолвинг имен, трассировка обычно идет быстрее.

-h N: максимальное число прыжков N, полезно для дальних направлений.

-w T: таймаут ожидания ответа от узла, миллисекунды.

Анализ задержек: большие значения RTT обычно появляются на магистральных и межконтинентальных участках. "Request timed out" чаще означает не разрыв связи, а то, что узел не отвечает на диагностические ICMP пакеты. Поэтому трассировка полезна как ориентир по маршруту, но не всегда показывает все реальные узлы.

Сравнение с Linux или macOS (теоретически): в traceroute часто используются UDP или ICMP Echo, формат вывода другой, а некоторые узлы могут отвечать иначе из-за фильтрации. Поэтому различия по числу прыжков и по видимым узлам возможны, даже если путь в целом похож.

флаги TRACERT помогают управлять скоростью и детализацией трассировки. По тестам видно, что на дальних направлениях задержка растет, а таймауты чаще связаны с фильтрацией ICMP на транзитных узлах.

Дополнительное задание 2. Попытка изменить маршрут, BGP и выводы

Во втором дополнительном задании я попробовал повлиять на маршрут до удаленного сервера так, чтобы трафик шел не через север Европы, а ближе к Польше. Идея была понятная: увидеть проблемный участок, заблокировать нежелательные транзитные узлы и проверить, перестроится ли путь. На практике все оказалось упрямее, чем хотелось.

Если говорить совсем коротко, BGP это протокол, через который крупные сети и провайдеры обмениваются маршрутами между автономными системами. То есть реальный межоператорский путь в Интернете выбирается не на клиентском ноутбуке, а выше: на оборудовании провайдера и его партнеров. Поэтому локальная Windows может ограничить соединение или оборвать его, но не обязана перестраивать глобальный маршрут так, как мне надо было.

Логи

Сначала я проверил, что базовые службы фильтрации Windows вообще работают нормально. Без этого дальше лезть в правила брандмауэра было бы бессмысленно.

```
PS C:\WINDOWS\System32> Get-Service MpsSvc,BFE | Format-Table Name,Status,StartType
Name    Status StartType
----    ------ ---------
BFE    Running Automatic
MpsSvc Running Automatic
```

Дальше я добавил правило блокировки для подсети 62.115.143.0/24. Это были как раз те адреса, которые встречались в трассе как транзитные узлы Telia. Логика была простая: если запретить трафик к этому диапазону, возможно, система или провайдер выберут другой путь.

```
PS C:\WINDOWS\System32> netsh advfirewall firewall add rule name="Block
62.115.143.0/24 OUT" dir=out action=block remoteip=62.115.143.0/24 profile=any
Ok.
```

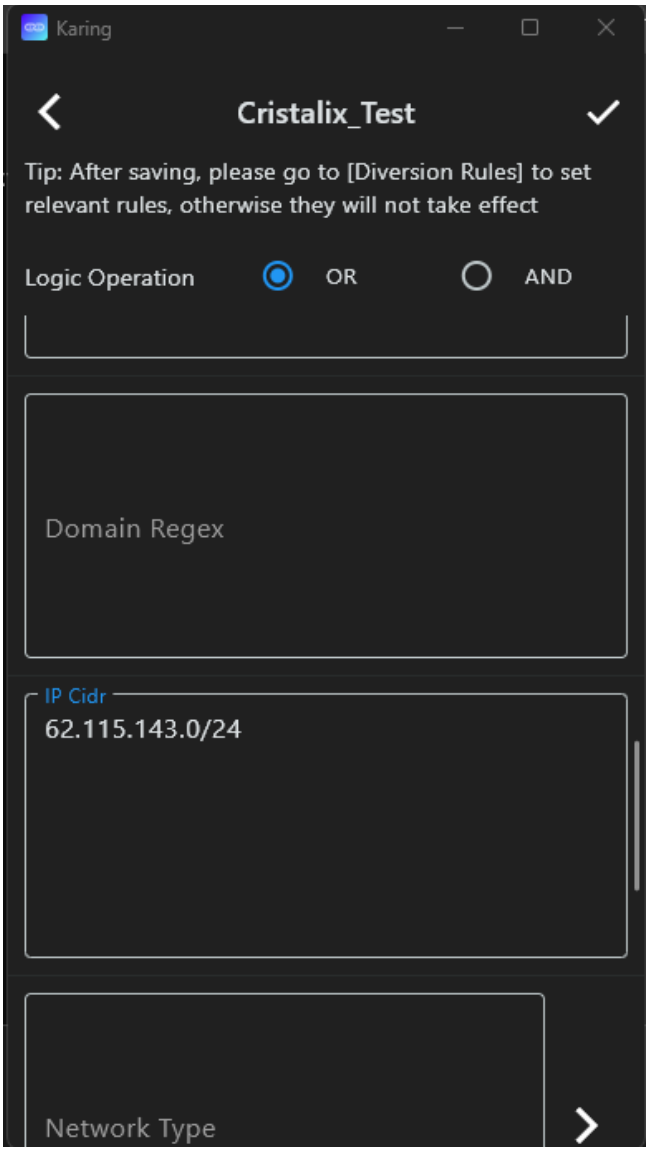

Рисунок 1. Правило блокировки диапазона адресов в приложении Karing

После этого я сначала по привычке попробовал команду traceroute в PowerShell, но Windows быстро напомнила, что здесь так не работает. Пришлось переходить на штатную tracert и уже через нее смотреть реальный путь до узла во Франции.

```
PS C:\WINDOWS\System32> traceroute 162.19.30.203
traceroute : The term 'traceroute' is not recognized as the name of a cmdlet,
function, script file, or operable
program. Check the spelling of the name, or if a path was included, verify that the
path is correct and try again.
PS C:\WINDOWS\System32> tracert -w 350 162.19.30.203
Tracing route to 58.gra3.ovh.abcd.network [162.19.30.203] over a maximum of 30 hops
1     1 ms     1 ms     1 ms  XiaoQiang [192.168.31.1]
2     5 ms     2 ms     2 ms  172.19.63.252
3     5 ms     2 ms     2 ms  10.200.40.9
4     5 ms     2 ms     2 ms  10.200.40.10
```

```
5     5 ms     2 ms     2 ms  176.213.132.18
6     3 ms     6 ms     2 ms  62.115.145.221
7     6 ms     3 ms     3 ms  62.115.145.220
8    25 ms    25 ms    25 ms  62.115.143.24
9    46 ms    46 ms     *     62.115.143.29
10   43 ms    47 ms    46 ms  62.115.124.117
11   49 ms    49 ms    50 ms  57.128.121.76
12    *        *        *     Request timed out.
```

По этой трассе уже стало видно главное: путь действительно идет через цепочку 62.115.x.x, то есть через те самые транзитные узлы, которые я хотел обойти. Дальше я отдельно проверил, что происходит, если обратиться к ним напрямую по TCP и ICMP.

```
PS C:\WINDOWS\System32> Test-NetConnection 62.115.143.24 -Port 443
PingSucceeded          : False
TcpTestSucceeded       : False
PS C:\WINDOWS\System32> Test-NetConnection 62.115.143.29 -Port 443
PingSucceeded          : False
TcpTestSucceeded       : False
PS C:\WINDOWS\System32> ping 62.115.143.24
General failure.
Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Потом я решил не гадать, а посмотреть, к каким адресам реально подключается нужное приложение. Для этого использовал связку Get-NetTCPConnection и Get-Process. Так получилось увидеть уже не транзитные hop из tracert, а настоящие конечные адреса, с которыми держится соединение.

```
PS C:\WINDOWS\System32> $p = (Get-Process java -ErrorAction SilentlyContinue).Id
PS C:\WINDOWS\System32> Get-NetTCPConnection -State Established |
>>   Where-Object { $_.OwningProcess -in $p } |
>>   Sort-Object RemoteAddress,RemotePort |
>>   Format-Table LocalAddress,LocalPort,RemoteAddress,RemotePort,OwningProcess -
Auto
192.168.31.15     21454 95.181.159.2       25565          6864
55681 95.181.159.3       25564         11968
54892 95.129.233.174        443         11968
54891 95.181.159.0        25565         11968
59886 8.6.112.0             443         11968
```

Здесь как раз и всплыл важный момент. Трассировка показывала промежуточные узлы 62.115.x.x, но само приложение общалось не с ними, а с конечными адресами вроде 95.181.159.3 и 95.129.233.174. То есть я пытался блокировать не конечную точку, а транзитный участок, который на уровне клиентской машины не является отдельным адресатом соединения.

После этого я добавил правила уже не просто на диапазон, а на конкретную программу java.exe. Смысл был такой: если обычное системное правило не

помогает, надо ограничить именно тот процесс, который устанавливает соединение.

```
PS C:\WINDOWS\System32> netsh advfirewall firewall add rule name="JAVA block Telia
FFM hop" dir=out action=block program="C:\Users\Ivan\.cristalix\updates\24-jre-win-
64\bin\java.exe" remoteip=62.115.0.0/16 profile=any
Ok.
PS C:\WINDOWS\System32> Get-NetTCPConnection -State Established |
>>   Where-Object { $_.OwningProcess -eq (Get-Process -Name java | Select-Object -
First 1).Id } |
>>   Format-Table LocalPort,RemoteAddress,RemotePort,OwningProcess -Auto
55840 95.181.159.3        25564         11968
37500 95.129.233.174        443         11968
```

Соединение после этого не исчезло, то есть сам путь не перестроился. Я продолжил проверку уже на живой трассировке к 95.181.159.3, чтобы увидеть, поменялась ли цепочка hop после добавления правил.

```
PS C:\WINDOWS\System32> tracert -w 200 95.181.159.3
Tracing route to 95.181.159.3 over a maximum of 30 hops
1     1 ms     1 ms     1 ms  192.168.31.1
2     2 ms     2 ms     2 ms  172.19.63.252
3     4 ms     2 ms     2 ms  10.200.40.9
4     5 ms     2 ms     2 ms  10.200.40.10
5     6 ms     2 ms     2 ms  176.213.132.18
6     6 ms     3 ms     6 ms  62.115.145.221
7     7 ms     4 ms     3 ms  62.115.145.220
8    25 ms    25 ms    25 ms  62.115.143.24
9    46 ms    46 ms    46 ms  62.115.143.29
10   49 ms    45 ms    46 ms  62.115.124.117
11   50 ms    49 ms    46 ms  57.128.121.76
15   56 ms    56 ms    53 ms  91.121.215.176
16   57 ms    53 ms    55 ms  37.59.16.32
```

Чтобы исключить вариант с направлением трафика, я добавил еще и входящее правило. Но и после этого маршрут визуально остался тем же: снова те же 62.115.x.x, тот же выход на 57.128.121.76 и дальше уже французские узлы.

```
PS C:\WINDOWS\System32> netsh advfirewall firewall add rule name="JAVA block Telia
FFM hop" dir=in action=block program="C:\Users\Ivan\.cristalix\updates\24-jre-win-
64\bin\java.exe" remoteip=62.115.0.0/16 profile=any
Ok.
PS C:\WINDOWS\System32> tracert -w 200 95.181.159.3
8    24 ms    26 ms    25 ms  62.115.143.24
9    45 ms    45 ms    47 ms  62.115.143.29
10   47 ms    46 ms    47 ms  62.115.124.117
11   48 ms     *       49 ms  57.128.121.76
15   53 ms    53 ms    53 ms  91.121.215.176
16   57 ms    57 ms    58 ms  37.59.16.32
```

Отдельно я проверил сами правила и убедился, что они действительно созданы и активны. То есть проблема была не в том, что команда не применилась, а именно в природе самой маршрутизации.

```
PS C:\WINDOWS\System32> netsh advfirewall firewall show rule name="JAVA block Telia
FFM hop" verbose
Enabled:                              Yes
Direction:                            In
RemoteIP:                             62.115.0.0/16
```

```
Program:                                C:\Users\Ivan\.cristalix\updates\24-jre-win-
64\bin\java.exe
Action:                                 Block
Enabled:                                Yes
Direction:                              Out
RemoteIP:                               62.115.0.0/16
Action:                                 Block
```

Следующий шаг был уже почти инженерный упрямецкий эксперимент: я сравнил трассировку до нескольких соседних адресов 95.181.159.0, 95.181.159.2 и 95.181.159.3. Хотел понять, не расходятся ли пути между серверами внутри одной подсети.

```
PS C:\WINDOWS\System32> tracert -d -w 200 95.181.159.0
8    26 ms    22 ms    26 ms  62.115.143.24
9    45 ms    43 ms    43 ms  62.115.143.29
10   45 ms    45 ms    46 ms  62.115.124.117
11   68 ms    49 ms    48 ms  57.128.121.76
15   56 ms    55 ms    53 ms  91.121.215.176
16   55 ms    54 ms    58 ms  37.59.16.28
PS C:\WINDOWS\System32> tracert -d -w 200 95.181.159.2
8    25 ms    25 ms    22 ms  62.115.143.24
9     *       42 ms    46 ms  62.115.143.29
10   46 ms    46 ms    43 ms  62.115.124.117
11   43 ms    44 ms    46 ms  57.128.121.78
16   53 ms    56 ms    53 ms  91.121.215.176
17   58 ms    56 ms    57 ms  37.59.16.36
PS C:\WINDOWS\System32> tracert -d -w 200 95.181.159.3
8    23 ms    26 ms    22 ms  62.115.143.24
9    46 ms    50 ms    46 ms  62.115.143.29
10   46 ms    44 ms    45 ms  62.115.124.117
11   50 ms     *       88 ms  57.128.121.76
15   56 ms    56 ms    53 ms  91.121.215.176
16   57 ms    56 ms    56 ms  37.59.16.32
```

Потом я попробовал самый прямолинейный вариант: руками добавить маршрут через адрес, который связывал с Польшей. Команда route add запись добавила, в таблице она действительно появилась. Но трассировка после этого осталась прежней.

```
PS C:\WINDOWS\System32> route add 95.181.159.3 mask 255.255.255.255 45.144.51.136
OK!
PS C:\WINDOWS\System32> route print 95.181.159.3
Network Destination        Netmask          Gateway       Interface  Metric
95.181.159.3  255.255.255.255    45.144.51.136    192.168.31.15     26
PS C:\WINDOWS\System32> tracert -d -w 200 95.181.159.3
8    23 ms    25 ms    22 ms  62.115.143.24
9    46 ms    45 ms    45 ms  62.115.143.29
10   46 ms    46 ms    46 ms  62.115.124.117
11   51 ms    47 ms    48 ms  57.128.121.76
15   54 ms    55 ms    55 ms  91.121.215.176
16   55 ms    59 ms    59 ms  37.59.16.32
```

Чтобы добить проверку, я еще запустил pathping. Он тоже показал ту же самую цепочку hop. При этом промежуточные внутренние адреса провайдера местами давали 100% потерь в ответах, но это не значило реальный обрыв канала. Это

был обычный случай, когда транзитные узлы не отвечают на ICMP так, как хочется диагностической утилите.

```
PS C:\WINDOWS\System32> pathping -n 95.181.159.3
0  192.168.31.15
1  192.168.31.1
2  172.19.63.252
3  10.200.40.9
4  10.200.40.10
5  176.213.132.18
6  62.115.145.221
7  62.115.145.220
8  62.115.143.24
9  62.115.143.29
10 62.115.124.117
11 57.128.121.76
Hop  RTT    Lost/Sent = Pct  Lost/Sent = Pct  Address
1    1ms     0/100 = 0%      0/100 = 0%      192.168.31.1
2   ---    100/100 =100%   100/100 =100%    172.19.63.252
6    5ms     0/100 = 0%      0/100 = 0%      62.115.145.221
8   24ms     0/100 = 0%      0/100 = 0%      62.115.143.24
10  46ms     0/100 = 0%      0/100 = 0%      62.115.124.117
11 ---     100/100 =100%     0/100 = 0%      57.128.121.76
```

В конце я сохранил краткую памятку по использованным командам, потому что в этом эксперименте их было много и каждая отвечала за свой кусок проверки. netsh advfirewall firewall add rule и delete rule нужны были для добавления и удаления правил. tracert и pathping показывали маршрут и поведение hop. Test-NetConnection и ping давали быструю проверку доступности, а Get-NetTCPConnection вместе с Get-Process помогали понять, с какими адресами реально держит связь приложение.

Рисунок 2. После блокировки соединение с сервером не устанавливается

По итогу этого дополнительного задания у меня получился довольно наглядный вывод. Локальный брандмауэр действительно может оборвать

конкретное соединение или помешать приложению работать нормально, но он не обязан перестраивать межпровайдерский путь до удаленного узла. Транзитные hop из tracert не равны конечным адресам, с которыми общается программа, и именно в этом была главная ловушка эксперимента.

Попытка направить трафик через Польшу через route add тоже не сработала как способ смены реального международного маршрута. Запись в таблице маршрутизации добавилась, но вся цепочка дальше по-прежнему определялась провайдером и его соседями по BGP. Поэтому для реального изменения внешнего пути нужна другая точка выхода в Интернет: другой провайдер, VPN с нужным регионом или собственное пограничное оборудование, где уже управляют маршрутизацией не на уровне клиента, а на уровне сети.

Общий вывод по работе

В ходе лабораторной работы я по очереди разобрал основные консольные средства Windows для сетевой диагностики и администрирования. Команда TRACERT показала, как пакет проходит через промежуточные маршрутизаторы и где по пути заметно растет задержка. PATHPING дополнил эту картину статистикой потерь и еще раз показал, что 100% loss на промежуточном узле не всегда означает реальную проблему передачи данных.

GETMAC помог отделить физические и виртуальные сетевые адаптеры и сравнить MAC-адреса Wi-Fi, Ethernet и VMware. Через NSLOOKUP получилось определить адрес роутера, увидеть ответ домашнего DNS и понять, почему сервер может отображаться как "UnKnown". По NETSTAT стало ясно, что анализ соединений надо всегда связывать с процессами и PID, а не просто смотреть на набор открытых портов.

NETSH и NET показали другую сторону Windows: сеть здесь тесно связана с правами пользователя, политиками безопасности и службами. То есть

команда сама по себе может быть верной, но без нужных разрешений или системного состояния результат будет совсем не тот, на который рассчитываешь. ROUTE.EXE дал уже более понятную картину выбора пути через метрику, а TELNET и файловые команды пригодились как простые утилиты для быстрой проверки порта и для работы с логами из консоли.

В дополнительной части с трассировкой хорошо проявилась разница между близкими и дальними направлениями. По маршрутам внутри России задержка оставалась небольшой и стабильной, а на направлениях в Азию и Австралию сразу было видно, где начинается магистральный и межконтинентальный участок. Это дало уже не абстрактное, а вполне живое понимание того, как расстояние и политика операторов влияют на сеть.

Самый интересный результат дал эксперимент с попыткой изменить маршрут до французского сервера. На практике выяснилось, что блокировка промежуточных адресов через Windows Firewall не меняет глобальную маршрутизацию по BGP. Я увидел, что одно дело это hop в трассировке, и совсем другое это реальные конечные адреса, к которым подключается приложение. Для меня это, наверное, и был главный практический итог всей лабораторной: сетевую картину надо читать на нескольких уровнях сразу, иначе очень легко бороться не с причиной, а только с ее следом.

МОСКВА 2026